

FASE LOAD — Dokumentasi Pembelajaran ETL Pipeline

Catatan: Dokumen ini ditulis dalam Bahasa Indonesia dengan istilah teknis dalam Bahasa Inggris. Dokumen ini adalah bagian dari perjalanan pembelajaran ETL (Extract, Transform, Load) Pipeline menggunakan Python. Source code yang dibahas berada di file `src/load.py` dan test file `tests/test_loads.py`.

Daftar Isi

1. Pendahuluan (Apa Itu Load?)
2. Arsitektur & Alur Kerja Load Phase
3. Star Schema — Penjelasan Mendalam
4. SQLAlchemy — Penjelasan Mendalam
5. `engine.connect()` vs `engine.begin()` — Perbedaan Lengkap
6. Fungsi `db_engine()` — Penjelasan
7. Fungsi `init_db()` — Penjelasan Baris Per Baris
8. Fungsi `load_dimensions()` — Penjelasan Baris Per Baris
9. Fungsi `load_fact()` — Penjelasan Baris Per Baris
10. Fungsi `load_data()` — Orchestrator Utama
11. Unit Testing — Penjelasan File Test
12. Konsep Database Yang Digunakan
13. Konsep Python Yang Digunakan
14. Idempotency — Konsep Penting dalam ETL
15. Ringkasan & Poin-Poin Penting

1. Pendahuluan (Apa Itu Load?)

1.1 Definisi Load dalam Konteks ETL

Dalam pipeline ETL (**Extract**, **Transform**, **Load**), fase **Load** adalah tahap **terakhir** di mana data yang sudah dibersihkan dan ditransformasi pada fase sebelumnya dipindahkan ke **penyimpanan akhir** (destination/target). Penyimpanan akhir ini biasanya berupa database relasional, data warehouse, atau data lake.

Bayangkan analogi sederhana:

| Analogi | ETL Phase | Penjelasan | |-----|-----|-----| | 🛒 Mengambil barang dari gudang | **Extract** |
 Mengambil data mentah dari sumber (file CSV, API, dsb.) | | ✂️ Membersihkan & menata barang | **Transform** |
 Membersihkan data, mengubah format, menghapus duplikat | | 📦 Menyimpan barang ke rak toko | **Load** | Menyimpan
 data bersih ke database/penyimpanan akhir |

1.2 Mengapa Load Adalah Fase Terakhir dan Sangat Kritis?

Load adalah fase **paling kritis** karena beberapa alasan:

1. **Data Integrity** — Jika proses load gagal di tengah jalan, database bisa memiliki data yang **tidak lengkap** (half-loaded state). Ini sangat berbahaya karena analisis bisnis akan menghasilkan insight yang salah.
2. **Irreversibility** — Setelah data masuk ke database dan di-commit, perubahan bersifat **permanen**. Berbeda dengan fase extract dan transform yang bekerja di memori (RAM), fase load menulis ke disk secara permanen.
3. **Relational Dependencies** — Dalam database relasional, tabel-tabel memiliki **hubungan** (relationship) melalui Foreign Key. Jika kita memasukkan data fact table sebelum dimension table terisi, akan terjadi error karena Foreign Key constraint dilanggar.
4. **Concurrency Issues** — Dalam production environment, mungkin ada beberapa proses yang menulis ke database secara bersamaan. Fase load harus menangani ini dengan benar melalui **transaksi database** (database transactions).

1.3 Tujuan Load

Tujuan utama fase Load dalam pipeline kita:

- ✅ Memindahkan data bersih (hasil transform) ke database relasional
- ✅ Membuat skema database (tabel-tabel) jika belum ada
- ✅ Memisahkan data ke dalam **Star Schema** (Dimension Tables + Fact Table)
- ✅ Mencegah **duplikasi data** jika pipeline dijalankan ulang (idempotency)
- ✅ Menangani error dengan baik menggunakan transaksi database (rollback jika gagal)

1.4 Jenis-Jenis Target Load

| Target | Penjelasan | Contoh | |-----|-----|-----| | **Database Relasional** | Database dengan struktur tabel yang ketat, mendukung SQL | PostgreSQL, MySQL, SQLite | | **Data Warehouse** | Database yang dioptimalkan untuk analisis (OLAP) | Google BigQuery, Amazon Redshift, Snowflake | | **Data Lake** | Penyimpanan data mentah dalam skala besar, format file | Amazon S3, Azure Data Lake, Google Cloud Storage | | **File** | Output berupa file | CSV, Parquet, JSON | | **NoSQL Database** | Database non-relasional | MongoDB, Cassandra, DynamoDB |

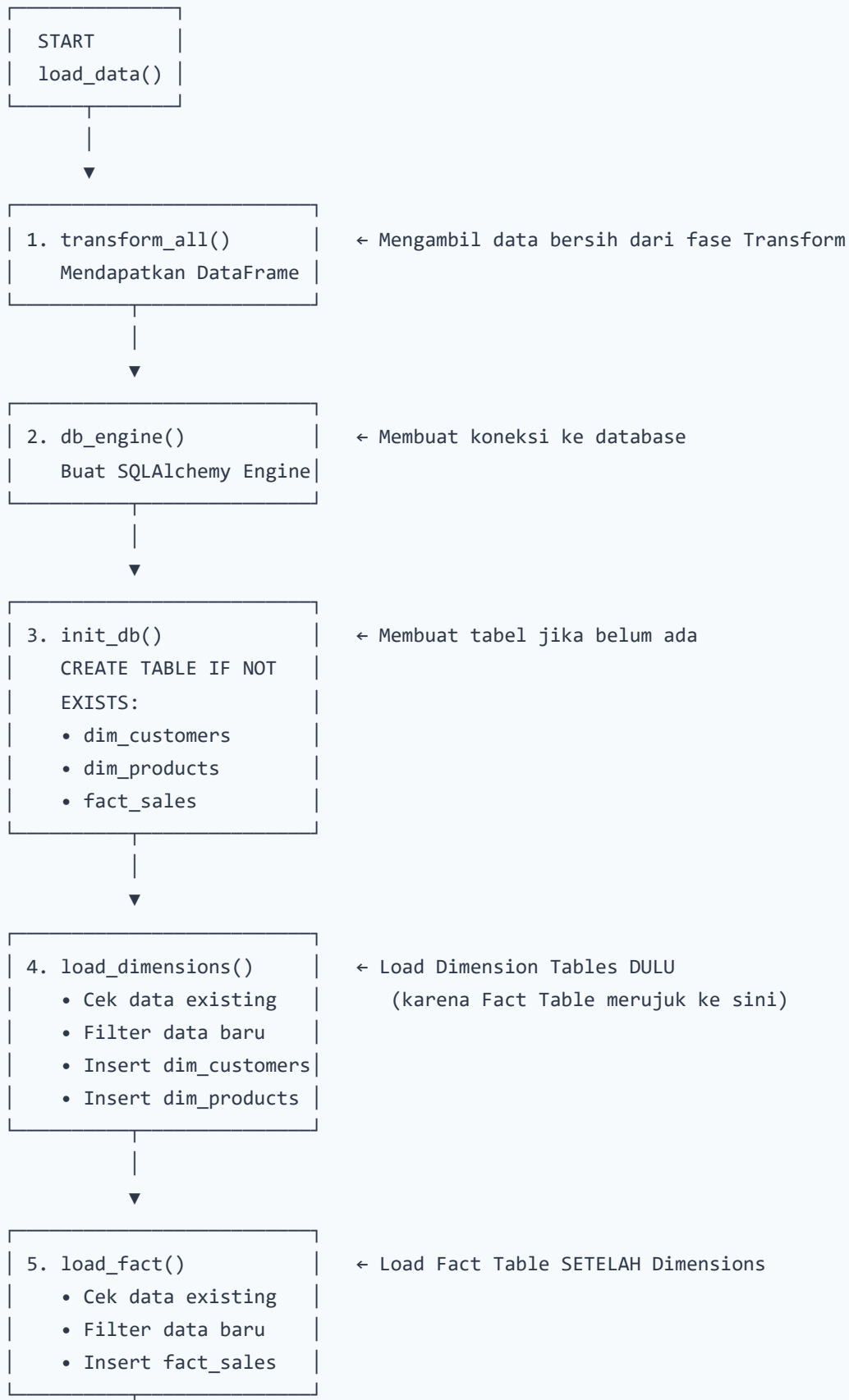
💡 Dalam project ini, kita menggunakan **database relasional** sebagai target load, dikelola melalui library **SQLAlchemy**.

2. **Arsitektur & Alur Kerja Load Phase**

2.1 Diagram Alur Load Phase

Berikut adalah diagram alur (flowchart) dari seluruh proses Load Phase dalam pipeline kita:

LOAD PHASE - ALUR KERJA





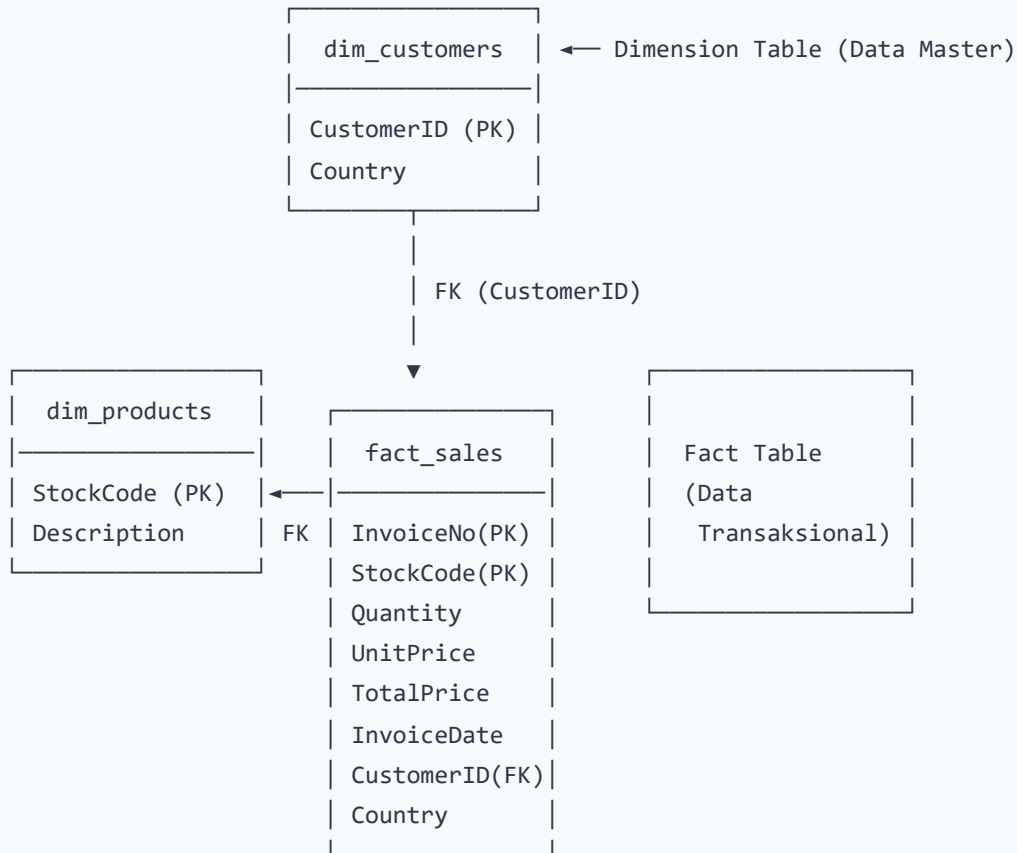
2.2 Penjelasan Alur

Perhatikan bahwa alur ini bersifat **sekuensial** (sequential) — artinya setiap langkah harus selesai sebelum langkah berikutnya dimulai. Ini sangat penting karena:

1. `transform_all()` harus selesai dulu agar kita punya DataFrame yang bersih.
2. `db_engine()` harus membuat koneksi database terlebih dahulu.
3. `init_db()` harus membuat tabel-tabel dulu sebelum kita bisa mengisi data.
4. `load_dimensions()` harus dijalankan **sebelum** `load_fact()` karena tabel `fact_sales` memiliki Foreign Key yang merujuk ke `dim_customers` dan `dim_products`. Jika dimension tables belum terisi, insert ke fact table akan gagal karena melanggar Foreign Key constraint.

2.3 Konsep Star Schema: Dimension Tables vs Fact Tables

Kode ini mengimplementasikan **Star Schema**, yaitu model data yang terdiri dari:



- **Dimension Tables** (`dim_*`): Menyimpan data **deskriptif/master** yang jarang berubah. Contoh: siapa customernya, apa produknya.
- **Fact Table** (`fact_*`): Menyimpan data **transaksional/metrik** yang terus bertambah. Contoh: penjualan yang terjadi.

3. ★ Star Schema — Penjelasan Mendalam

3.1 Apa Itu Star Schema dan Mengapa Digunakan?

Star Schema adalah arsitektur desain database yang digunakan secara luas dalam **data warehousing** dan **business intelligence**. Dinamakan "bintang" karena jika digambarkan, bentuknya menyerupai bintang — satu tabel fakta di tengah dikelilingi oleh beberapa tabel dimensi.

Mengapa Star Schema digunakan?

| Alasan | Penjelasan | |-----|-----| | 🚀 **Performa query cepat** | Struktur yang sederhana memungkinkan database mengeksekusi JOIN dengan efisien | | 📊 **Mudah untuk analisis** | Analis data dapat menulis query SQL yang intuitif | | 🗄️ **Mengurangi redundansi** | Data master (customer, product) disimpan satu kali di dimension table, bukan

diulang di setiap baris transaksi || 🛠 **Standar industri** | Hampir semua data warehouse menggunakan variasi Star Schema || 🧩 **Mudah di-extend** | Menambah dimensi baru tidak memerlukan perubahan besar |

3.2 Dimension Tables — Data Deskriptif/Master

dim_customers — Tabel Dimensi Pelanggan

```
CREATE TABLE IF NOT EXISTS dim_customers (
  CustomerID INT PRIMARY KEY,
  Country VARCHAR(50)
)
```

Tabel ini menyimpan **data unik pelanggan**. Setiap pelanggan hanya muncul **sekali** di tabel ini, teridentifikasi oleh **CustomerID**.

Kolom	Tipe Data	Constraint	Penjelasan
CustomerID	INT	PRIMARY KEY	ID unik pelanggan, tidak boleh duplikat dan tidak boleh NULL
Country	VARCHAR(50)		Negara asal pelanggan, maksimal 50 karakter

dim_products — Tabel Dimensi Produk

```
CREATE TABLE IF NOT EXISTS dim_products (
  StockCode VARCHAR(20) PRIMARY KEY,
  Description VARCHAR(100)
)
```

Tabel ini menyimpan **katalog produk unik**. Setiap produk hanya muncul **sekali** berdasarkan **StockCode**.

Kolom	Tipe Data	Constraint	Penjelasan
StockCode	VARCHAR(20)	PRIMARY KEY	Kode stok produk, unik untuk setiap produk
Description	VARCHAR(100)		Deskripsi produk, maksimal 100 karakter

3.3 Fact Table — Data Transaksional/Metrik

fact_sales — Tabel Fakta Penjualan

```
CREATE TABLE IF NOT EXISTS fact_sales (  
    InvoiceNo VARCHAR(20),  
    StockCode VARCHAR(20),  
    Quantity INT,  
    UnitPrice FLOAT,  
    TotalPrice FLOAT,  
    InvoiceDate DATETIME,  
    CustomerID INT,  
    Country VARCHAR(50),  
    PRIMARY KEY (InvoiceNo, StockCode),  
    FOREIGN KEY (CustomerID) REFERENCES dim_customers(CustomerID),  
    FOREIGN KEY (StockCode) REFERENCES dim_products(StockCode)  
)
```

Tabel ini menyimpan **setiap transaksi penjualan**. Ini adalah "jantung" dari data kita — berisi semua metrik bisnis (quantity, price) dan referensi ke dimension tables.

Kolom	Tipe Data	Constraint	Penjelasan	InvoiceNo	VARCHAR(20)	PRIMARY KEY (komposit)	Nomor invoice/faktur	StockCode	VARCHAR(20)	PRIMARY KEY (komposit), FOREIGN KEY	Kode produk, merujuk ke dim_products	Quantity	INT	-	Jumlah barang yang dibeli	UnitPrice	FLOAT	-	Harga per unit barang	TotalPrice	FLOAT	-	Total harga (Quantity × UnitPrice)	InvoiceDate	DATETIME	-	Tanggal dan waktu transaksi	CustomerID	INT	FOREIGN KEY	ID pelanggan, merujuk ke dim_customers	Country	VARCHAR(50)	-	Negara tempat transaksi terjadi
-------	-----------	------------	------------	-----------	-------------	------------------------	----------------------	-----------	-------------	-------------------------------------	--------------------------------------	----------	-----	---	---------------------------	-----------	-------	---	-----------------------	------------	-------	---	------------------------------------	-------------	----------	---	-----------------------------	------------	-----	-------------	--	---------	-------------	---	---------------------------------

3.4 PRIMARY KEY — Apa Itu dan Mengapa Penting?

PRIMARY KEY adalah constraint (batasan) yang diterapkan pada satu atau lebih kolom dalam tabel database. PRIMARY KEY memiliki dua aturan utama:

- UNIQUE** — Nilai di kolom tersebut **tidak boleh duplikat**. Tidak ada dua baris yang bisa memiliki nilai PRIMARY KEY yang sama.
- NOT NULL** — Nilai di kolom tersebut **tidak boleh kosong** (NULL).

Contoh di dim_customers:

CustomerID	Country
12345	UK
67890	France
12345	Germany
NULL	Spain

← CustomerID adalah PRIMARY KEY

✓ OK

✓ OK

✗ GAGAL! CustomerID 12345 sudah ada (duplikat)

✗ GAGAL! CustomerID tidak boleh NULL

Mengapa penting? PRIMARY KEY menjamin bahwa setiap baris dalam tabel dapat **diidentifikasi secara unik**. Ini seperti KTP — setiap orang punya nomor KTP yang berbeda.

3.5 FOREIGN KEY — Apa Itu dan Cara Kerjanya

FOREIGN KEY adalah constraint yang **menghubungkan** satu tabel dengan tabel lain. FOREIGN KEY di tabel anak (child) harus merujuk ke PRIMARY KEY di tabel induk (parent).

FOREIGN KEY (CustomerID) REFERENCES dim_customers(CustomerID)

Artinya: "Nilai **CustomerID** di tabel **fact_sales** **harus sudah ada** di kolom **CustomerID** tabel **dim_customers**."

dim_customers (Parent)

CustomerID	Country
12345	UK
67890	France

fact_sales (Child)

InvoiceNo	StockCode	CustomerID
INV001	85123A	12345
INV002	71053	67890
INV003	22423	99999

✓ OK

✓ OK

✗ GAGAL!

↑

CustomerID 99999 TIDAK ADA di dim_customers

Mengapa penting?

- ✓ Menjaga **referential integrity** (integritas referensi) — data selalu konsisten
- ✓ Mencegah **orphan records** — baris di fact table yang merujuk ke customer yang tidak ada
- ✓ Itulah mengapa `load_dimensions()` harus dijalankan **SEBELUM** `load_fact()`

3.6 COMPOSITE PRIMARY KEY — Kapan Digunakan

Composite Primary Key adalah PRIMARY KEY yang terdiri dari **dua atau lebih kolom**. Kombinasi nilai dari kolom-kolom tersebut harus unik.

PRIMARY KEY (InvoiceNo, StockCode)

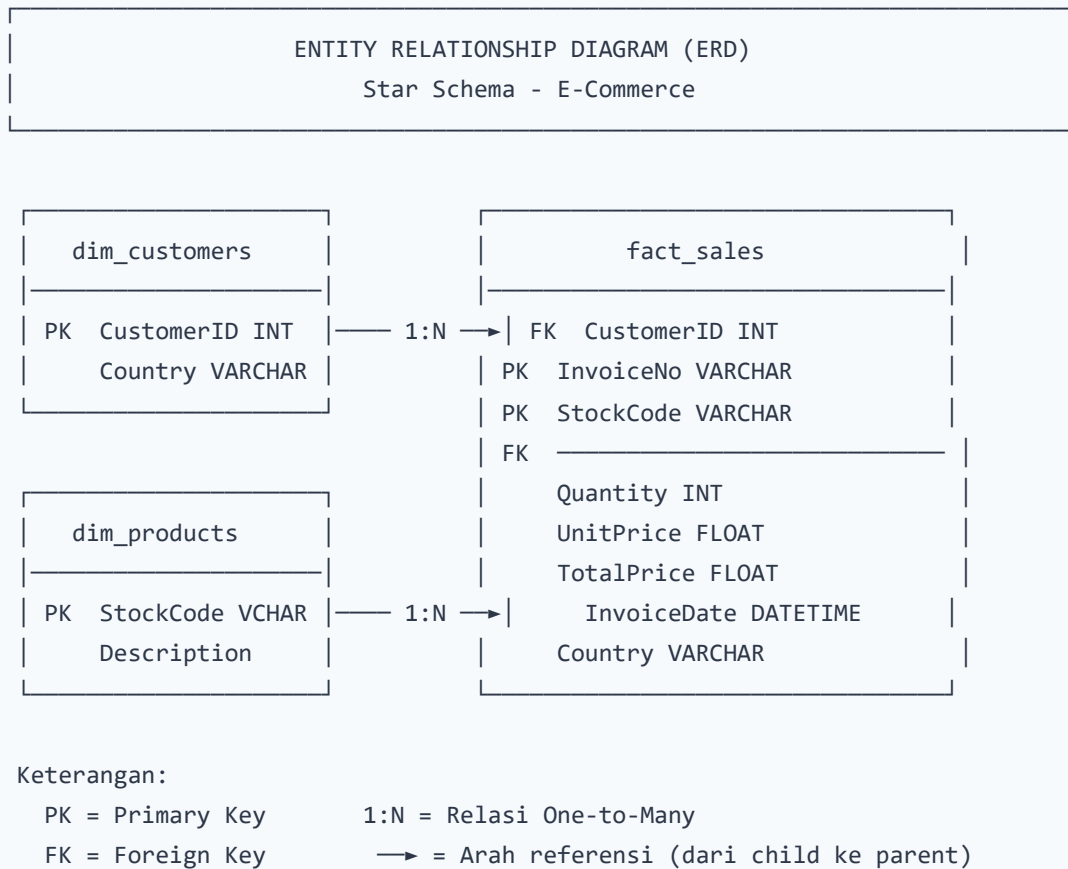
Artinya: Kombinasi **InvoiceNo** + **StockCode** harus unik. Satu invoice **boleh** memiliki banyak produk, dan satu produk **boleh** muncul di banyak invoice, tapi **satu invoice dengan satu produk yang sama** hanya boleh muncul **sekali**.

InvoiceNo	StockCode	Quantity	
INV001	85123A	6	✓ OK
INV001	71053	3	✓ OK (InvoiceNo sama, StockCode beda)
INV002	85123A	2	✓ OK (StockCode sama, InvoiceNo beda)
INV001	85123A	1	✗ GAGAL! Kombinasi INV001+85123A sudah ada

Kapan menggunakan Composite Primary Key?

- Ketika **satu kolom saja tidak cukup** untuk mengidentifikasi baris secara unik
- Dalam kasus kita: Satu invoice bisa berisi banyak item produk, jadi kita butuh kombinasi **InvoiceNo** + **StockCode**

3.7 Diagram Relasi Antar Tabel (Entity Relationship Diagram)



Penjelasan relasi:

- dim_customers** → **fact_sales** (1:N / One-to-Many): Satu customer bisa memiliki **banyak** transaksi penjualan.
- dim_products** → **fact_sales** (1:N / One-to-Many): Satu produk bisa muncul di **banyak** transaksi penjualan.

4. 🛠️ SQLAlchemy — Penjelasan Mendalam

4.1 Apa Itu SQLAlchemy?

SQLAlchemy adalah library Python yang menyediakan **abstraksi** untuk berinteraksi dengan database relasional. Alih-alih menulis kode yang spesifik untuk setiap jenis database (PostgreSQL, MySQL, SQLite, dll.), kita cukup menggunakan satu API dari SQLAlchemy.

Tanpa SQLAlchemy:

Python Code
psycopg2 (PG)
mysql-connector
sqlite3
PostgreSQL
MySQL
SQLite

Dengan SQLAlchemy:

Python Code	
SQLAlchemy	← Satu library untuk semua database
(abstraksi)	
Driver:	
psycopg2 / mysql	
/ sqlite3	
PostgreSQL /	
MySQL / SQLite	

4.2 Mengapa Menggunakan SQLAlchemy (Bukan Raw SQL Driver)?

| Fitur | Raw SQL Driver | SQLAlchemy | |-----|-----|-----| | **Portabilitas** | Kode terikat pada satu database | Ganti database hanya dengan mengubah connection string | | **Connection Pooling** | Harus diimplementasikan manual | Sudah built-in | | **Transaction Management** | Manual | Terintegrasi (context manager) | | **Integrasi Pandas** | Perlu kode tambahan | `to_sql()` dan `read_sql()` langsung bekerja | | **ORM Support** | Tidak ada | Tersedia jika dibutuhkan |

4.3 ORM vs Core — Dua Pendekatan SQLAlchemy

SQLAlchemy memiliki **dua lapisan** yang bisa digunakan:

◆ SQLAlchemy Core (Digunakan di kode ini)

SQLAlchemy Core adalah pendekatan **low-level** yang bekerja langsung dengan SQL expressions dan connections. Kita menulis SQL secara eksplisit.

```
# Contoh penggunaan Core (seperti di kode kita)
with engine.connect() as conn:
    conn.execute(text("SELECT * FROM dim_customers"))
```

◆ SQLAlchemy ORM (Object Relational Mapper)

SQLAlchemy ORM adalah pendekatan **high-level** di mana tabel database direpresentasikan sebagai Python class, dan baris data direpresentasikan sebagai Python object.

```
# Contoh penggunaan ORM (TIDAK digunakan di kode ini, hanya sebagai perbandingan)
from sqlalchemy.orm import Session
from models import Customer

with Session(engine) as session:
    customer = session.query(Customer).filter_by(CustomerID=12345).first()
    print(customer.Country)  # "UK"
```

Mengapa kode kita menggunakan Core?

-  Lebih sederhana untuk ETL pipeline
-  Kita sudah menggunakan Pandas untuk manipulasi data, jadi ORM tidak diperlukan
-  `to_sql()` dari Pandas terintegrasi langsung dengan SQLAlchemy Engine/Connection
-  Performanya lebih baik untuk bulk insert

4.4 `create_engine()` — Apa yang Dilakukan?

```
from sqlalchemy import create_engine

def db_engine(connection_str: str):
    return create_engine(connection_str)
```

Fungsi `create_engine()` membuat sebuah **Engine object** — ini adalah titik awal interaksi dengan database. Engine **TIDAK** langsung membuka koneksi ke database, melainkan:

1. **Mem-parse connection string** — Menentukan jenis database, host, port, username, password, dan nama database.
2. **Membuat Connection Pool** — Mempersiapkan pool koneksi yang dapat digunakan ulang.
3. **Memilih SQL dialect** — SQLAlchemy otomatis memilih "bahasa" SQL yang tepat untuk database target.

4.5 Connection String Format

Format connection string mengikuti standar URI:

```
dialect+driver://username:password@host:port/database_name
```

Contoh untuk berbagai database:

Database	Connection String	----- -----	SQLite (file lokal)	<code>sqlite:///data/ecommerce.db</code>	
PostgreSQL	<code>postgresql://user:password@localhost:5432/ecommerce</code>		MySQL		

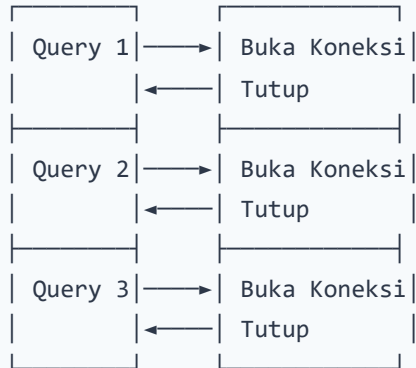
```
mysql+pymysql://user:password@localhost:3306/ecommerce || SQL Server |
```

```
mssql+pyodbc://user:password@localhost/ecommerce |
```

4.6 Connection Pool — Apa Itu dan Mengapa Penting?

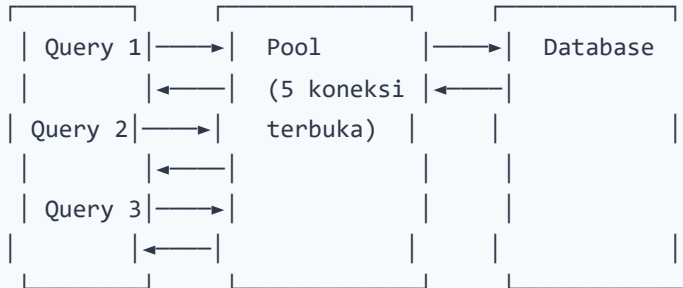
Connection Pool adalah mekanisme di mana SQLAlchemy menjaga **sekumpulan koneksi database yang terbuka** dan siap digunakan. Alih-alih membuka dan menutup koneksi baru setiap kali kita perlu query ke database (yang mahal/lambat), kita **meminjam** koneksi dari pool dan **mengembalikannya** setelah selesai.

Tanpa Connection Pool:



3x buka/tutup = LAMBAT

Dengan Connection Pool:



Koneksi di-reuse = CEPAT

SQLAlchemy secara default membuat pool dengan **5 koneksi** yang siap digunakan. Konfigurasi pool bisa diubah melalui parameter `create_engine()`.

5. 🏴‍☠️ engine.connect() vs engine.begin() — Perbedaan Lengkap

Ini adalah salah satu konsep **paling penting** yang harus dipahami. Kedua method ini digunakan di kode kita, dan memiliki perilaku yang **sangat berbeda**.

5.1 engine.connect() — Membuka Koneksi Tanpa Transaksi Otomatis

```

with engine.connect() as conn:
    # conn adalah Connection object
    # TIDAK ada transaksi yang otomatis dimulai
    # Kita harus MANUAL memanggil conn.begin(), conn.commit(), conn.rollback()
    pass
  
```

Ketika menggunakan `engine.connect()`:

- Kita mendapatkan **Connection object**
- **Tidak ada transaksi** yang otomatis dimulai
- Kita harus **secara manual** memanggil `conn.begin()` untuk memulai transaksi
- Kita harus **secara manual** memanggil `conn.commit()` untuk menyimpan perubahan
- Jika terjadi error, kita harus **secara manual** memanggil `conn.rollback()` untuk membatalkan

5.2 `engine.begin()` — Membuka Koneksi DENGAN Transaksi Otomatis

```
with engine.begin() as conn:
    # conn adalah Connection object yang sudah DALAM transaksi
    # Transaksi otomatis di-COMMIT jika blok with selesai tanpa error
    # Transaksi otomatis di-ROLLBACK jika terjadi exception
    pass
```

Ketika menggunakan `engine.begin()` :

- Kita mendapatkan **Connection object yang sudah berada dalam transaksi**
- Transaksi otomatis **COMMIT** jika blok `with` selesai tanpa error
- Transaksi otomatis **ROLLBACK** jika terjadi exception di dalam blok `with`
- Kita **TIDAK perlu** memanggil `commit()` atau `rollback()` secara manual

5.3 Transaksi Database — ACID Properties

Sebelum membahas lebih lanjut, pahami dulu apa itu **transaksi database** dan prinsip **ACID**:

Transaksi adalah sekelompok operasi database yang diperlakukan sebagai **satu unit kerja**. Semua operasi dalam transaksi harus **berhasil semua** atau **gagal semua**.

Property	Nama Lengkap	Penjelasan	Contoh	----- ----- ----- -----	A	Atomicity	Semua operasi berhasil, atau tidak ada yang berhasil
							Jika insert ke <code>dim_customers</code> berhasil tapi insert ke <code>fact_sales</code> gagal, maka insert ke <code>dim_customers</code> juga dibatalkan
					C	Consistency	Database selalu dalam keadaan valid sebelum dan sesudah transaksi
							Foreign Key constraint selalu terpenuhi
					I	Isolation	Transaksi yang berjalan bersamaan tidak saling mengganggu
							Dua pipeline ETL yang jalan bersamaan tidak menyebabkan data corrupt
					D	Durability	Setelah commit, data tersimpan permanen meskipun sistem crash
							Setelah <code>commit()</code> , data aman di disk

5.4 `commit()` — Menyimpan Perubahan Secara Permanen

```
conn.commit() # Semua perubahan yang dilakukan sejak begin() sekarang PERMANEN
```

`commit()` memberitahu database: "Semua operasi yang sudah saya lakukan dalam transaksi ini sudah benar. Simpan semua perubahan ini secara permanen ke disk."

5.5 `rollback()` — Membatalkan Perubahan Jika Ada Error

```
conn.rollback() # Semua perubahan yang dilakukan sejak begin() DIBATALKAN
```

`rollback()` memberitahu database: "Ada yang salah. Batalkan semua operasi yang sudah saya lakukan dalam transaksi ini. Kembalikan database ke keadaan sebelum transaksi dimulai."

5.6 Context Manager (`with` Statement) — Auto-Close Connection

`with` statement di Python digunakan sebagai **context manager** yang memastikan resource (dalam hal ini, koneksi database) **otomatis ditutup** setelah blok kode selesai, baik karena selesai normal maupun karena exception.

```
# Tanpa context manager (BERBAHAYA):
conn = engine.connect()
conn.execute(text("SELECT * FROM dim_customers"))
conn.close() # Bagaimana jika ada error di baris sebelumnya? conn.close() tidak pernah dipanggil!

# Dengan context manager (AMAN):
with engine.connect() as conn:
    conn.execute(text("SELECT * FROM dim_customers"))
# conn.close() otomatis dipanggil, bahkan jika ada exception!
```

5.7 Tabel Perbandingan Lengkap

Aspek	<code>engine.connect()</code>		<code>engine.begin()</code>		----- ----- -----		Transaksi otomatis?	✗
Tidak	✓ Ya		Perlu <code>conn.begin()</code> ?		✓ Ya, jika butuh transaksi		✗ Tidak, sudah otomatis	
	Perlu <code>conn.commit()</code> ?		✓ Ya, manual		✗ Tidak, otomatis saat keluar <code>with</code>		Perlu <code>conn.rollback()</code> ?	✓ Ya, dalam
blok	<code>except</code>		✗ Tidak, otomatis saat exception		Auto-close koneksi?		✓ Ya (via <code>with</code>)	✓ Ya (via <code>with</code>)
Cocok untuk	Operasi yang butuh kontrol manual (DDL) Operasi DML yang harus atomic							
Digunakan di kode ini	<code>init_db()</code> <code>load_dimensions()</code> , <code>load_fact()</code>							

5.8 Contoh Kode untuk Kedua Pendekatan

Pendekatan 1: `engine.connect()` + Manual Transaction (digunakan di `init_db()`)

```
def init_db(engine):
    with engine.connect() as conn:      # Buka koneksi (tanpa transaksi)
        conn.begin()                   # Mulai transaksi SECARA MANUAL
        try:
            conn.execute("""...""")    # Jalankan DDL
            conn.commit()               # Simpan perubahan SECARA MANUAL
        except Exception as e:
            conn.rollback()             # Batalkan perubahan SECARA MANUAL
            raise                       # Re-raise exception
```

Pendekatan 2: `engine.begin()` + Auto Transaction (digunakan di `load_dimensions()`)

```
def load_dimensions(df, engine):
    with engine.begin() as conn:      # Buka koneksi + mulai transaksi OTOMATIS
        conn.execute(text("..."))     # Jalankan query
    # Jika selesai tanpa error → otomatis COMMIT
    # Jika ada exception → otomatis ROLLBACK
```

5.9 Kapan Menggunakan Yang Mana?

| Situasi | Gunakan | Alasan | |-----|-----|-----| | DDL (CREATE TABLE, ALTER TABLE) | `engine.connect()` | Kontrol manual lebih fleksibel untuk DDL statements | | DML sederhana (INSERT, UPDATE) | `engine.begin()` | Lebih ringkas, auto-commit/rollback | | Multiple DML yang harus atomic | `engine.begin()` | Semua operasi dalam satu transaksi otomatis | | Operasi read-only (SELECT) | Keduanya bisa | Tapi `engine.connect()` lebih umum untuk read |

6. Fungsi `db_engine()` — Penjelasan

6.1 Kode Lengkap

```
def db_engine(connection_str: str):
    return create_engine(connection_str)
```

6.2 Penjelasan Detail

Fungsi ini sangat sederhana — hanya **membungkus** (wrapper) fungsi `create_engine()` dari SQLAlchemy. Fungsi ini menerima satu parameter:

- `connection_str` (tipe `str`): Connection string yang berisi informasi koneksi database.

Mengapa membuat wrapper?

1. **Abstraksi** — Jika di masa depan kita perlu menambahkan konfigurasi tambahan ke engine (seperti pool size, echo mode, dll.), kita hanya perlu mengubah fungsi ini.
2. **Testability** — Lebih mudah di-mock saat testing.

6.3 Contoh Connection String untuk Berbagai Database

```
# SQLite (file lokal - cocok untuk development/testing)
engine = db_engine("sqlite:///data/ecommerce.db")

# SQLite (in-memory - untuk testing cepat)
engine = db_engine("sqlite:///memory:")

# PostgreSQL (production)
engine = db_engine("postgresql://admin:secret123@localhost:5432/ecommerce")

# MySQL
engine = db_engine("mysql+pymysql://admin:secret123@localhost:3306/ecommerce")
```

 **Peringatan Keamanan:** Jangan pernah hardcode password di dalam source code. Gunakan **environment variables** atau file `.env` untuk menyimpan kredensial.

7. Fungsi `init_db()` — Penjelasan Baris Per Baris

7.1 Kode Lengkap

```
def init_db(engine):
    with engine.connect() as conn:
        conn.begin()
        try:
            conn.execute("""
            CREATE TABLE IF NOT EXISTS dim_customers (
                CustomerID INT PRIMARY KEY,
                Country VARCHAR(50)
            )
            """)

            conn.execute("""
            CREATE TABLE IF NOT EXISTS dim_products (
                StockCode VARCHAR(20) PRIMARY KEY,
                Description VARCHAR(100)
            )
            """)

            conn.execute("""
            CREATE TABLE IF NOT EXISTS fact_sales (
                InvoiceNo VARCHAR(20),
                StockCode VARCHAR(20),
                Quantity INT,
                UnitPrice FLOAT,
                TotalPrice FLOAT,
                InvoiceDate DATETIME,
                CustomerID INT,
                Country VARCHAR(50),
                PRIMARY KEY (InvoiceNo, StockCode),
                FOREIGN KEY (CustomerID) REFERENCES dim_customers(CustomerID),
                FOREIGN KEY (StockCode) REFERENCES dim_products(StockCode)
            )
            """)
            conn.commit()
            logger.info("Database initialized successfully")
        except Exception as e:
            logger.error(f"Error initializing database: {str(e)}")
            conn.rollback()
            raise
```

7.2 Penjelasan Baris Per Baris

Baris 1: `def init_db(engine):`

Mendefinisikan fungsi yang menerima satu parameter — **engine** (SQLAlchemy Engine object). Fungsi ini bertanggung jawab membuat semua tabel yang dibutuhkan di database.

Baris 2: `with engine.connect() as conn:`

Membuka koneksi ke database menggunakan **context manager**. Variabel `conn` adalah **Connection object**. Menggunakan `engine.connect()` karena kita membutuhkan kontrol manual atas transaksi untuk operasi DDL (CREATE TABLE).

Baris 3: `conn.begin()`

Memulai transaksi database secara **manual**. Semua operasi setelah baris ini akan menjadi bagian dari satu transaksi. Perubahan belum tersimpan ke database sampai `commit()` dipanggil.

Baris 4: `try:`

Memulai blok **try-except** untuk menangani kemungkinan error. Jika salah satu CREATE TABLE gagal, kita akan menangkap error-nya di blok `except`.

Baris 5-9: CREATE TABLE `dim_customers`

```
CREATE TABLE IF NOT EXISTS dim_customers (  
    CustomerID INT PRIMARY KEY,  
    Country VARCHAR(50)  
)
```

- **CREATE TABLE** — Perintah DDL untuk membuat tabel baru.
- **IF NOT EXISTS** — Klausa yang mencegah error jika tabel sudah ada. Jika tabel `dim_customers` sudah ada, perintah ini **tidak melakukan apa-apa** (dilewati). Ini membuat kode kita **idempotent** — aman dijalankan berkali-kali.
- **CustomerID INT PRIMARY KEY** — Kolom `CustomerID` bertipe **INT** (integer/bilangan bulat) dan merupakan PRIMARY KEY.
- **Country VARCHAR(50)** — Kolom `Country` bertipe **VARCHAR(50)** — string dengan panjang maksimal 50 karakter.

Baris 10-14: CREATE TABLE `dim_products`

```
CREATE TABLE IF NOT EXISTS dim_products (  
    StockCode VARCHAR(20) PRIMARY KEY,  
    Description VARCHAR(100)  
)
```

Sama seperti `dim_customers`, tapi untuk data produk. `StockCode` adalah PRIMARY KEY bertipe `VARCHAR(20)` karena kode stok bisa berisi huruf dan angka (contoh: `"85123A"`).

Baris 15-27: CREATE TABLE `fact_sales`

```
CREATE TABLE IF NOT EXISTS fact_sales (  
    InvoiceNo VARCHAR(20),  
    StockCode VARCHAR(20),  
    Quantity INT,  
    UnitPrice FLOAT,  
    TotalPrice FLOAT,  
    InvoiceDate DATETIME,  
    CustomerID INT,  
    Country VARCHAR(50),  
    PRIMARY KEY (InvoiceNo, StockCode),  
    FOREIGN KEY (CustomerID) REFERENCES dim_customers(CustomerID),  
    FOREIGN KEY (StockCode) REFERENCES dim_products(StockCode)  
)
```

- `PRIMARY KEY (InvoiceNo, StockCode)` — Composite Primary Key dari dua kolom.
- `FOREIGN KEY (CustomerID) REFERENCES dim_customers(CustomerID)` — Menghubungkan kolom `CustomerID` di `fact_sales` ke kolom `CustomerID` di `dim_customers`.
- `FOREIGN KEY (StockCode) REFERENCES dim_products(StockCode)` — Menghubungkan kolom `StockCode` di `fact_sales` ke kolom `StockCode` di `dim_products`.

Baris 28: `conn.commit()`

Menyimpan semua perubahan (pembuatan tabel) secara **permanen** ke database. Jika `commit()` tidak dipanggil, tabel-tabel yang dibuat tidak akan tersimpan.

Baris 29: `logger.info("Database initialized successfully")`

Mencatat pesan log bahwa inisialisasi database berhasil.

Baris 30-33: Blok `except`

```
except Exception as e:
    logger.error(f"Error initializing database: {str(e)}")
    conn.rollback()
    raise
```

- `except Exception as e:` — Menangkap semua jenis exception.
- `logger.error(...)` — Mencatat pesan error ke log.
- `conn.rollback()` — Membatalkan SEMUA perubahan yang sudah dilakukan sejak `conn.begin()`. Jika tabel pertama berhasil dibuat tapi tabel kedua gagal, tabel pertama **juga dibatalkan**.
- `raise` — Melempar kembali exception yang tertangkap ke level yang lebih tinggi. Ini penting agar caller function tahu bahwa `init_db()` gagal.

7.3 Tipe Data SQL yang Digunakan

Tipe Data SQL	Penjelasan	Contoh Nilai	Padanan Python
INT	Bilangan bulat	12345, -7	int
VARCHAR(n)	String dengan panjang maksimal n karakter	"UK", "85123A"	str
FLOAT	Bilangan desimal (floating point)	2.55, 15.30	float
DATETIME	Tanggal dan waktu	2010-12-01 08:26:00	datetime.datetime

7.4 Mengapa Menggunakan `engine.connect()` + Manual Transaction di Sini?

Fungsi `init_db()` menggunakan `engine.connect()` dengan manual transaction (`conn.begin()`, `conn.commit()`, `conn.rollback()`) alih-alih `engine.begin()`. Ini memberikan kita:

1. **Kontrol eksplisit** atas kapan transaksi dimulai dan berakhir
2. **Logging yang lebih baik** — kita bisa log error spesifik sebelum rollback
3. **Fleksibilitas** — bisa menambahkan logika tambahan antara `begin()` dan `commit()`

8. Fungsi `load_dimensions()` — Penjelasan Baris Per Baris

8.1 Kode Lengkap

```
def load_dimensions(df: pd.DataFrame, engine):
    logger.info("Loading Dimensions Tables...")

    # Extract unique customers
    dim_customers = df[["CustomerID", "Country"]].dropna(subset=["CustomerID"]).drop_duplicates(subset=['CustomerID'])
    dim_customers["CustomerID"] = dim_customers["CustomerID"].astype(int)

    # Extract unique products
    dim_products = df[["StockCode", "Description"]].dropna(subset=["StockCode"]).drop_duplicates(subset=["StockCode"])

    with engine.begin() as conn:
        exists_customers_raw = conn.execute(text("SELECT CustomerID FROM dim_customers")).fetchall()
        exists_customers = {row[0] for row in exists_customers_raw}

        exists_products_raw = conn.execute(text("SELECT StockCode FROM dim_products")).fetchall()
        exists_products = {row[0] for row in exists_products_raw}

        new_customers = dim_customers[~dim_customers["CustomerID"].isin(exists_customers)]
        new_products = dim_products[~dim_products["StockCode"].isin(exists_products)]

        if not new_customers.empty:
            new_customers.to_sql("dim_customers", conn, if_exists="append", index=False)
            logger.info(f"Loaded {len(new_customers)} new customers")
        else:
            logger.info("No new customers to load")

        if not new_products.empty:
            new_products.to_sql("dim_products", conn, if_exists="append", index=False)
            logger.info(f"Loaded {len(new_products)} new products")
        else:
            logger.info("No new products to load")
```

8.2 Penjelasan Baris Per Baris

Baris 1: `def load_dimensions(df: pd.DataFrame, engine):`

Mendefinisikan fungsi dengan dua parameter:

- `df` (tipe `pd.DataFrame`): DataFrame yang berisi data bersih hasil transformasi. Type hint `pd.DataFrame` memberi tahu bahwa parameter ini harus berupa Pandas DataFrame.
- `engine`: SQLAlchemy Engine object untuk koneksi database.

Baris 2: `logger.info("Loading Dimensions Tables...")`

Mencatat pesan log bahwa proses loading dimension tables dimulai. Ini penting untuk **monitoring** dan **debugging** — kita bisa melihat di log kapan proses ini berjalan.

Baris 4-5: Memilih dan Menyiapkan Data Customer

```
dim_customers = df[["CustomerID", "Country"]].dropna(subset=["CustomerID"]).drop_duplicates(subset=["CustomerID"])
dim_customers["CustomerID"] = dim_customers["CustomerID"].astype(int)
```

Baris ini melakukan **tiga operasi** secara berantai (method chaining):

Operasi 1: `df[["CustomerID", "Country"]]` — Memilih Kolom

```
# Dari DataFrame lengkap yang mungkin berisi 8+ kolom:
# InvoiceNo | StockCode | Description | Quantity | UnitPrice | TotalPrice | InvoiceDate | CustomerID | Country

# Kita hanya mengambil 2 kolom yang relevan untuk dim_customers:
dim_customers = df[["CustomerID", "Country"]]

# Hasil:
# CustomerID | Country
# 12345      | UK
# 12345      | UK      ← duplikat!
# 67890      | France
# NaN       | Germany ← CustomerID kosong!
```

💡 Kenapa pakai double bracket `[[ ]]` ?

- `df["CustomerID"]` → Mengembalikan **Series** (satu kolom)
- `df[["CustomerID", "Country"]]` → Mengembalikan **DataFrame** (subset kolom) Kita butuh DataFrame karena dimension table memiliki beberapa kolom.

Operasi 2: `.dropna(subset=["CustomerID"])` — Menghapus Baris dengan NaN


```
# Menghapus semua baris di mana CustomerID bernilai NaN (kosong/null)
# Parameter subset=["CustomerID"] berarti: hanya cek NaN di kolom CustomerID saja

# Sebelum dropna:
# CustomerID | Country
# 12345      | UK
# 67890      | France
# NaN       | Germany ← akan dihapus

# Sesudah dropna:
# CustomerID | Country
# 12345      | UK
# 67890      | France
```

Mengapa perlu menghapus NaN? Karena `CustomerID` adalah **PRIMARY KEY** di tabel `dim_customers`. PRIMARY KEY **tidak boleh NULL**. Jika kita mencoba insert baris dengan `CustomerID = NaN`, database akan menolak dengan error.

Operasi 3: `.drop_duplicates(subset=["CustomerID"])` — Menghapus Duplikat

```
# Menghapus baris duplikat berdasarkan kolom CustomerID
# Jika ada beberapa baris dengan CustomerID yang sama, hanya baris PERTAMA yang dipertahankan

# Sebelum drop_duplicates:
# CustomerID | Country
# 12345      | UK
# 12345      | UK ← duplikat, akan dihapus
# 67890      | France

# Sesudah drop_duplicates:
# CustomerID | Country
# 12345      | UK
# 67890      | France
```

Mengapa perlu menghapus duplikat? Karena `CustomerID` adalah PRIMARY KEY, kita **tidak bisa** insert dua baris dengan `CustomerID` yang sama. Jika kita mencoba, database akan menolak dengan error `UNIQUE constraint violated`.

Baris tambahan: `.astype(int)` — Konversi Tipe Data

```
dim_customers["CustomerID"] = dim_customers["CustomerID"].astype(int)
```

Mengubah tipe data kolom `CustomerID` dari `float64` (default Pandas ketika ada NaN di kolom numeric) ke `int`. Ini diperlukan karena:

- Pandas menyimpan kolom numeric yang mengandung NaN sebagai `float64` (karena `int` tidak bisa menyimpan NaN)
- Setelah kita `dropna()`, tidak ada lagi NaN, jadi aman untuk konversi ke `int`
- Database mengharapkan tipe `INT`, bukan `FLOAT`

```
Sebelum astype(int): CustomerID = [12345.0, 67890.0] ← float
Setelah astype(int): CustomerID = [12345, 67890]      ← int
```

Baris 8-9: Memilih dan Menyiapkan Data Produk

```
dim_products = df[["StockCode", "Description"]].dropna(subset=["StockCode"]).drop_duplicates(subset=["StockCode"])
```

Operasi yang sama seperti customer, tapi untuk data produk:

1. Pilih kolom `StockCode` dan `Description`
2. Hapus baris dengan `StockCode` yang kosong (NaN)
3. Hapus duplikat berdasarkan `StockCode`

💡 Tidak perlu `astype(int)` karena `StockCode` bertipe `VARCHAR` (string), bukan `INT`.

Baris 11: `with engine.begin() as conn:`

Membuka koneksi database **dengan transaksi otomatis**. Semua operasi di dalam blok `with` ini akan berada dalam **satu transaksi**:

- Jika semua operasi berhasil → otomatis **COMMIT**
- Jika ada exception → otomatis **ROLLBACK**

Baris 12-13: Mengambil Data Customer yang Sudah Ada di Database

```
exists_customers_raw = conn.execute(text("SELECT CustomerID FROM dim_customers")).fetchall()
exists_customers = {row[0] for row in exists_customers_raw}
```

`conn.execute(text("SELECT CustomerID FROM dim_customers"))` — Menjalankan query SQL untuk mengambil semua `CustomerID` yang **sudah ada** di database.

💡 **Mengapa perlu `text()` ?** Pada SQLAlchemy versi 2.0+, semua raw SQL string **harus** dibungkus dalam fungsi `text()` dari `sqlalchemy`. Ini karena:

1. **Keamanan** — `text()` menandai string sebagai SQL yang sah, mencegah SQL injection
2. **Kompatibilitas** — SQLAlchemy 2.0 tidak lagi menerima raw string langsung di `execute()`
3. **Eksplisit** — Membuat kode lebih jelas bahwa kita sedang menjalankan raw SQL

`.fetchall()` — Mengambil **semua** hasil query sebagai list of tuples.

```
# Jika database sudah berisi:
# CustomerID: 12345, 67890

# fetchall() mengembalikan:
exists_customers_raw = [(12345,), (67890,)]
# Setiap row adalah tuple, meskipun hanya satu kolom
```

`{row[0] for row in exists_customers_raw}` — **Set Comprehension**

```
# Mengubah list of tuples menjadi set of values
exists_customers = {row[0] for row in exists_customers_raw}

# Langkah per langkah:
# exists_customers_raw = [(12345,), (67890,)]
# row[0] mengambil elemen pertama dari setiap tuple:
# 12345, 67890
# { } membungkusnya dalam set:
# exists_customers = {12345, 67890}
```

💡 **Mengapa menggunakan `set` (bukan `list`)?**

| Operasi | List | Set | |-----|-----|-----| | Cek keanggotaan (`x in collection`) | O(n) — lambat | O(1) — sangat cepat | | Contoh: 100,000 customer IDs | Cek setiap elemen satu per satu | Langsung tahu jawabannya |

Karena kita akan mengecek apakah setiap CustomerID di DataFrame **sudah ada** di database, menggunakan `set` jauh lebih efisien — terutama untuk data besar.

Baris 15-16: Mengambil Data Produk yang Sudah Ada

```
exists_products_raw = conn.execute(text("SELECT StockCode FROM dim_products")).fetchall()
exists_products = {row[0] for row in exists_products_raw}
```

Operasi yang sama seperti customer, tapi untuk produk.

Baris 18: Memfilter Customer Baru (Belum Ada di Database)

```
new_customers = dim_customers[~dim_customers["CustomerID"].isin(exists_customers)]
```

Baris ini adalah salah satu yang **paling penting** dalam logika deduplikasi. Mari kita pecahkan:

Langkah 1: `dim_customers["CustomerID"].isin(exists_customers)`

`.isin()` mengecek apakah setiap nilai di kolom `CustomerID` **ada di dalam** set `exists_customers`.

```
# dim_customers["CustomerID"] = [12345, 67890, 11111]
# exists_customers = {12345, 67890}

# .isin(exists_customers) menghasilkan boolean Series:
# [True, True, False]
#   ↑      ↑      ↑
# 12345 67890 11111
# sudah  sudah  BELUM
# ada   ada   ada
```

Langkah 2: Operator `~` (tilde/negasi)

Operator `~` adalah operator **bitwise NOT** yang dalam konteks Pandas digunakan sebagai **negasi boolean** — membalik `True` menjadi `False` dan sebaliknya.

```
# Sebelum ~: [True, True, False]
# Sesudah ~: [False, False, True]
#           ↑      ↑      ↑
#           12345 67890 11111
#           skip  skip  AMBIL!
```

Langkah 3: Boolean Indexing `dim_customers[...]`

Menggunakan boolean Series sebagai "filter" untuk DataFrame. Hanya baris dengan nilai `True` yang akan dipilih.

```
# dim_customers:
# CustomerID | Country
# 12345      | UK          ← False → dibuang (sudah ada di DB)
# 67890      | France      ← False → dibuang (sudah ada di DB)
# 11111      | Germany     ← True  → DIAMBIL (belum ada di DB)

# new_customers:
# CustomerID | Country
# 11111      | Germany     ← Hanya customer baru!
```

Baris 19: Memfilter Produk Baru

```
new_products = dim_products[~dim_products["StockCode"].isin(exists_products)]
```

Logika yang sama seperti customer, tapi untuk produk.

Baris 21-24: Insert Customer Baru ke Database

```
if not new_customers.empty:
    new_customers.to_sql("dim_customers", conn, if_exists="append", index=False)
    logger.info(f"Loaded {len(new_customers)} new customers")
else:
    logger.info("No new customers to load")
```

`if not new_customers.empty:` — Mengecek apakah DataFrame `new_customers` **tidak kosong**. Properti `.empty` mengembalikan `True` jika DataFrame tidak memiliki baris. `not` membalikinya — jadi `if not new_customers.empty` berarti "jika ada data baru".

Mengapa perlu dicek? Karena memanggil `to_sql()` pada DataFrame kosong:

1. Tidak berguna (membuang resource)
2. Bisa menyebabkan error pada beberapa database/driver

```
new_customers.to_sql("dim_customers", conn, if_exists="append", index=False)
```

Ini adalah method Pandas yang sangat penting — `to_sql()` menulis DataFrame ke tabel SQL.

| Parameter | Nilai | Penjelasan | |-----|-----|-----| | `name` (positional) | `"dim_customers"` | Nama tabel tujuan di database | | `con` | `conn` | Connection object SQLAlchemy | | `if_exists` | `"append"` | Apa yang dilakukan jika tabel sudah ada: `"append"` = tambahkan baris baru | | `index` | `False` | Jangan sertakan index DataFrame sebagai kolom di tabel SQL |

Opsi `if_exists` :

| Nilai | Perilaku | |-----|-----| | `"fail"` | Raise error jika tabel sudah ada (default) | | `"replace"` | Hapus tabel lama, buat tabel baru, insert semua data | | `"append"` | Tambahkan baris baru ke tabel yang sudah ada |

⚠ **Jangan pernah gunakan `"replace"` di production!** Ini akan menghapus seluruh data yang sudah ada di tabel.

`index=False` — Penting! Tanpa ini, Pandas akan menyertakan index DataFrame (0, 1, 2, 3, ...) sebagai kolom tambahan di tabel SQL, yang tidak kita inginkan.

Baris 26-30: Insert Produk Baru ke Database

```
if not new_products.empty:
    new_products.to_sql("dim_products", conn, if_exists="append", index=False)
    logger.info(f"Loaded {len(new_products)} new products")
else:
    logger.info("No new products to load")
```

Logika yang sama seperti customer.

9. Fungsi `load_fact()` — Penjelasan Baris Per Baris

9.1 Kode Lengkap

```
def load_fact(df: pd.DataFrame, engine):
    logger.info("Loading Fact Table...")

    fact_sales = df[
        ["InvoiceNo", "StockCode", "Quantity", "UnitPrice", "TotalPrice", "InvoiceDate", "CustomerID", "Count"]
    ].dropna(subset=["CustomerID"])
    fact_sales["CustomerID"] = fact_sales["CustomerID"].astype(int)

    with engine.begin() as conn:
        exists_sales_raw = conn.execute(text("SELECT InvoiceNo, StockCode FROM fact_sales")).fetchall()
        exists_sales = {(str(row[0]), str(row[1])) for row in exists_sales_raw}

        sales_tuples = list(zip(fact_sales["InvoiceNo"].astype(str), fact_sales["StockCode"].astype(str)))
        new_sales = fact_sales[[t not in exists_sales for t in sales_tuples]]

        if not new_sales.empty:
            new_sales.to_sql("fact_sales", conn, if_exists="append", index=False)
            logger.info(f"Loaded {len(new_sales)} new sales")
        else:
            logger.info("No new sales to load")
```

9.2 Penjelasan Baris Per Baris

Baris 3-6: Memilih dan Menyiapkan Data Fakta

```
fact_sales = df[
    ["InvoiceNo", "StockCode", "Quantity", "UnitPrice", "TotalPrice", "InvoiceDate", "CustomerID", "Count"]
].dropna(subset=["CustomerID"])
fact_sales["CustomerID"] = fact_sales["CustomerID"].astype(int)
```

- Memilih **8 kolom** yang diperlukan untuk fact table
- Menghapus baris dengan `CustomerID` kosong (karena `CustomerID` adalah Foreign Key ke `dim_customers`)
- Mengkonversi `CustomerID` ke integer

💡 **Mengapa dropna berdasarkan CustomerID ?** Karena `fact_sales` memiliki `FOREIGN KEY (CustomerID) REFERENCES dim_customers(CustomerID)`. Jika `CustomerID` adalah `NULL/NaN`, insert akan gagal karena Foreign Key constraint tidak terpenuhi (atau menghasilkan orphan record).

Baris 9-10: Mengambil Data Penjualan yang Sudah Ada — Composite Key

```
exists_sales_raw = conn.execute(text("SELECT InvoiceNo, StockCode FROM fact_sales")).fetchall()
exists_sales = {(str(row[0]), str(row[1])) for row in exists_sales_raw}
```

Perbedaan penting dengan `load_dimensions()` !

Di `load_dimensions()`, kita hanya mengecek **satu kolom** (single key):

```
exists_customers = {row[0] for row in exists_customers_raw}
# Hasil: {12345, 67890} ← set of single values
```

Di `load_fact()`, kita mengecek **dua kolom** (composite key):

```
exists_sales = {(str(row[0]), str(row[1])) for row in exists_sales_raw}
# Hasil: {("INV001", "85123A"), ("INV001", "71053"), ("INV002", "85123A")}
#           ↑ set of tuples ↑
```

Mengapa `str()` ? Untuk memastikan konsistensi tipe data. Nilai dari database mungkin berbeda tipenya dengan nilai di DataFrame, sehingga konversi ke string memastikan perbandingan yang benar.

Set of Tuples — Setiap elemen dalam set adalah **tuple** berisi dua nilai `(InvoiceNo, StockCode)`. Ini karena PRIMARY KEY di `fact_sales` adalah composite key dari kedua kolom.

```
# Ilustrasi set of tuples:
exists_sales = {
    ("INV001", "85123A"), # Transaksi INV001 untuk produk 85123A
    ("INV001", "71053"), # Transaksi INV001 untuk produk 71053
    ("INV002", "85123A"), # Transaksi INV002 untuk produk 85123A
}
```


Baris 12: `zip()` — Menggabungkan Dua Kolom Menjadi Tuple

```
sales_tuples = list(zip(fact_sales["InvoiceNo"].astype(str), fact_sales["StockCode"].astype(str)))
```

Fungsi `zip()` menggabungkan elemen-elemen dari dua atau lebih iterable menjadi **pasangan-pasangan (tuples)**.

```
# Visualisasi zip():
# fact_sales["InvoiceNo"] = ["INV001", "INV001", "INV002", "INV003"]
# fact_sales["StockCode"] = ["85123A", "71053", "85123A", "22423"]

# zip() menggabungkan elemen pada posisi yang sama:
# zip(InvoiceNo, StockCode) =
#   ("INV001", "85123A") ← posisi 0
#   ("INV001", "71053")  ← posisi 1
#   ("INV002", "85123A") ← posisi 2
#   ("INV003", "22423")  ← posisi 3

# list() mengubah zip object menjadi list:
sales_tuples = [
    ("INV001", "85123A"),
    ("INV001", "71053"),
    ("INV002", "85123A"),
    ("INV003", "22423")
]
```

Mengapa `.astype(str)` ? Untuk memastikan tipe data yang konsisten dengan `exists_sales` yang juga di-convert ke string.

Baris 13: List Comprehension untuk Boolean Mask

```
new_sales = fact_sales[[t not in exists_sales for t in sales_tuples]]
```

Ini adalah baris yang **paling kompleks** di seluruh kode. Mari pecahkan:

Langkah 1: List comprehension `[t not in exists_sales for t in sales_tuples]`

```
# sales_tuples = [("INV001", "85123A"), ("INV001", "71053"), ("INV002", "85123A"), ("INV003", "22423")]
# exists_sales = {"INV001", "85123A"}, {"INV001", "71053"}, {"INV002", "85123A"}

# Untuk setiap tuple t dalam sales_tuples, cek apakah t TIDAK ADA di exists_sales:
# ("INV001", "85123A") not in exists_sales → False (sudah ada)
# ("INV001", "71053") not in exists_sales → False (sudah ada)
# ("INV002", "85123A") not in exists_sales → False (sudah ada)
# ("INV003", "22423") not in exists_sales → True (BELUM ada! ← baru)

# Hasil: [False, False, False, True]
```

Langkah 2: Boolean Indexing `fact_sales[...]`

```
# Menggunakan list boolean sebagai filter:
# fact_sales:
# InvoiceNo | StockCode | Quantity | ...
# INV001    | 85123A    | 6        | ... ← False → dibuang
# INV001    | 71053     | 3        | ... ← False → dibuang
# INV002    | 85123A    | 2        | ... ← False → dibuang
# INV003    | 22423     | 1        | ... ← True  → DIAMBIL

# new_sales:
# InvoiceNo | StockCode | Quantity | ...
# INV003    | 22423     | 1        | ... ← Hanya data baru!
```

9.3 Perbedaan Deduplikasi: Single Key vs Composite Key

Aspek	Single Key (Dimensions)	Composite Key (Facts)	Key
Satu kolom	(<code>CustomerID</code> atau <code>StockCode</code>)	Dua kolom (<code>InvoiceNo</code> + <code>StockCode</code>)	Existing data <code>set</code> of single values <code>set</code> of tuples
Contoh	<code>{12345, 67890}</code>	<code>{("INV001", "85123A"), ...}</code>	Filter method <code>~df["col"].isin(set)</code> <code>[t not in set for t in tuples]</code>
Mengapa berbeda	<code>.isin()</code> hanya mendukung single value	Composite key perlu perbandingan tuple	

10. 🎯 Fungsi `load_data()` — Orchestrator Utama

10.1 Kode Lengkap

```
def load_data(file_path: str, connection_str: str):
    logger.info(f"Loading data from {file_path}...")
    df = transform_all(file_path)
    engine = db_engine(connection_str)
    init_db(engine)
    load_dimensions(df, engine)
    load_fact(df, engine)
    logger.info("Data loaded successfully")
```

10.2 Penjelasan Baris Per Baris

Fungsi `load_data()` adalah **orchestrator** — fungsi yang mengatur dan menjalankan seluruh proses Load phase dalam urutan yang benar.

Baris	Kode	Penjelasan
1	<code>def load_data(file_path: str, connection_str: str):</code>	Menerima path file data dan connection string database
2	<code>logger.info(f"Loading data from {file_path}...")</code>	Log awal proses
3	<code>df = transform_all(file_path)</code>	Integrasi dengan Transform phase — memanggil fungsi dari <code>src/transform.py</code> untuk mendapatkan DataFrame yang sudah bersih
4	<code>engine = db_engine(connection_str)</code>	Membuat SQLAlchemy Engine
5	<code>init_db(engine)</code>	Membuat tabel-tabel jika belum ada
6	<code>load_dimensions(df, engine)</code>	Load dimension tables DULU
7	<code>load_fact(df, engine)</code>	Load fact table SETELAH dimensions
8	<code>logger.info("Data loaded successfully")</code>	Log akhir proses

10.3 Alur Lengkap dari File Path hingga Data Loaded

```
load_data("data/raw/ecommerce.csv", "sqlite:///data/ecommerce.db")
|
|→ transform_all("data/raw/ecommerce.csv")
|   |
|   |→ extract_get_data()      ← Baca file CSV
|   |→ fix_data_type()        ← Perbaiki tipe data
|   |→ handle_missing_values() ← Tangani data kosong
|   |→ remove_duplicates()    ← Hapus duplikat
|   |→ add_total_price()      ← Tambah kolom TotalPrice
|   |→ return df (DataFrame bersih)
|
|→ db_engine("sqlite:///data/ecommerce.db")
|   |→ return engine (SQLAlchemy Engine)
|
|→ init_db(engine)
|   |→ CREATE TABLE IF NOT EXISTS (3 tabel)
|
|→ load_dimensions(df, engine)
|   |→ Siapkan dim_customers (dropna, drop_duplicates)
|   |→ Siapkan dim_products (dropna, drop_duplicates)
|   |→ Cek data existing di database
|   |→ Filter hanya data baru
|   |→ Insert data baru ke database
|
|→ load_fact(df, engine)
|   |→ Siapkan fact_sales (dropna, astype)
|   |→ Cek data existing (composite key)
|   |→ Filter hanya data baru
|   |→ Insert data baru ke database
```

10.4 Integrasi dengan Transform Phase

Perhatikan baris:

```
from src.transform import transform_all
```

dan:

```
df = transform_all(file_path)
```

Fungsi `transform_all()` dari module `src/transform.py` menjalankan **seluruh proses Transform** dan mengembalikan DataFrame yang sudah bersih. Ini menunjukkan bagaimana fase-fase ETL **terhubung satu sama lain** — output dari Transform menjadi input untuk Load.

11. Unit Testing — Penjelasan File Test

11.1 Apa Itu Unit Testing dan Mengapa Penting?

Unit testing adalah praktik menulis kode **tes otomatis** yang memverifikasi bahwa setiap "unit" (biasanya satu fungsi) dalam program kita berperilaku **sesuai harapan**.

Mengapa penting dalam ETL?

| Alasan | Penjelasan | |-----|-----| |  **Mencegah bug** | Mendeteksi masalah sebelum kode masuk production | |  **Refactoring aman** | Bisa mengubah kode dengan percaya diri karena ada tes yang memverifikasi | |  **Dokumentasi hidup** | Test menunjukkan bagaimana fungsi seharusnya digunakan | |  **Menghemat waktu** | Mendeteksi error dalam hitungan detik, bukan jam debugging manual |

11.2 Kode Test Lengkap

```

import pandas as pd
import sys
import logging
from pathlib import Path
project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))
from unittest.mock import MagicMock
from src.load import load_dimensions, load_fact

logger = logging.getLogger(__name__)

def test_load_dimensions_basic():
    engine = MagicMock()
    mock_conn = engine.begin.return_value.__enter__.return_value
    mock_conn.execute.return_value.fetchall.side_effect = [[], []]
    df = pd.DataFrame({
        "CustomerID": [1, 1, 2],
        "Country": ["USA", "USA", "UK"],
        "StockCode": ["65828E", "84987B", "90730C"],
        "Description": ["ProdA", "ProdA", "ProdB"]
    })
    with pd.option_context("mode.chained_assignment", None):
        load_dimensions(df, engine)
    assert mock_conn.execute.call_count==2

def test_loads_facts_basic():
    engine = MagicMock()
    mock_conn = engine.begin.return_value.__enter__.return_value
    mock_conn.execute.return_value.fetchall.side_effect = [[]]
    df = pd.DataFrame({
        "InvoiceNo": [1, 1, 2],
        "StockCode": ["65828E", "84987B", "90730C"],
        "Quantity": [1, 1, 2],
        "UnitPrice": [1, 1, 2],
        "TotalPrice": [1, 1, 2],
        "InvoiceDate": ["2022-01-01", "2022-01-01", "2022-01-01"],
        "CustomerID": [1, 1, 2],
        "Country": ["USA", "USA", "UK"]
    })
    with pd.option_context("mode.chained_assignment", None):
        load_fact(df, engine)
    assert mock_conn.execute.call_count==1

```

11.3 MagicMock — Apa Itu Mock Object?

Mock object adalah objek "tiruan" yang **berpura-pura** menjadi objek asli. Dalam konteks testing, kita menggunakan mock untuk **menggantikan** komponen yang sulit atau tidak mungkin digunakan dalam tes — seperti **database**.

Kode asli (production):

```
load_dimensions()
↓
engine.begin()
↓
conn.execute(SQL)
↓
DATABASE ASLI
PostgreSQL/SQLite
Harus berjalan!
```

Kode test (mock):

```
load_dimensions()
↓
engine.begin()
↓
conn.execute(SQL)
↓
MOCK (pura-pura)
Tidak perlu DB
Instan & gratis!
```

Mengapa menggunakan mock?

1. ⚡ **Cepat** — Tidak perlu koneksi database asli
2. 🗝️ **Terisolasi** — Tes tidak tergantung pada state database
3. 🎯 **Terkontrol** — Kita bisa mengatur persis apa yang dikembalikan oleh mock
4. 🌐 **Portable** — Tes bisa dijalankan di mana saja tanpa setup database

MagicMock adalah class dari module `unittest.mock` bawaan Python. **MagicMock** secara otomatis membuat mock untuk:

- Semua attribute (`.foo` , `.bar`)
- Semua method calls (`.execute()` , `.fetchall()`)
- Semua magic methods (`__enter__` , `__exit__`)

```
engine = MagicMock()
# engine.connect()      → mengembalikan MagicMock baru
# engine.begin()        → mengembalikan MagicMock baru
# engine.anything()     → mengembalikan MagicMock baru
# Semua otomatis, tidak perlu setup satu per satu!
```

11.4 `engine.begin().return_value.__enter__.return_value` — Apa Artinya?

Ini adalah bagian yang paling **membingungkan** bagi pemula. Mari kita pecahkan langkah demi langkah:

Kode asli di `load_dimensions()` :

```
with engine.begin() as conn:
    conn.execute(text("SELECT CustomerID FROM dim_customers"))
```

Apa yang terjadi di balik layar ketika `with engine.begin() as conn:` dieksekusi:

<code>with engine.begin() as conn:</code>	
<code> — engine.begin()</code>	→ memanggil method <code>begin()</code>
<code> — return_value</code>	→ objek yang dikembalikan oleh <code>begin()</code>
<code> — .__enter__()</code>	→ method yang dipanggil oleh 'with' (context manager)
<code> — return_value</code>	→ ini yang menjadi variable 'conn'
<code> — as conn</code>	→ 'conn' = hasil dari <code>__enter__()</code>

Jadi dalam testing:

```
engine = MagicMock()
mock_conn = engine.begin.return_value.__enter__.return_value

# Artinya:
# engine.begin()          → MagicMock (return_value dari begin)
# .__enter__()            → MagicMock (dipanggil oleh 'with')
# .return_value           → mock_conn (menjadi variabel 'conn' di kode asli)
```

Sekarang, ketika kode asli menjalankan `conn.execute(...)`, `conn` sebenarnya adalah `mock_conn` — objek mock yang kita kendalikan!

11.5 `side_effect` — Mensimulasikan Return Values yang Berbeda

```
mock_conn.execute.return_value.fetchall.side_effect = [[], []]
```

`side_effect` memungkinkan kita mengatur **urutan return values** untuk pemanggilan berulang dari fungsi yang sama.


```
# Pemanggilan pertama .fetchall() → mengembalikan [] (tidak ada customer existing)
# Pemanggilan kedua .fetchall() → mengembalikan [] (tidak ada product existing)

# Ini mensimulasikan database yang KOSONG – belum ada data customer maupun product
```

Mengapa `[], []` ?

Karena di `load_dimensions()`, `.fetchall()` dipanggil **DUA kali**:

1. Pertama: `conn.execute(text("SELECT CustomerID FROM dim_customers")).fetchall()` → `[]`
2. Kedua: `conn.execute(text("SELECT StockCode FROM dim_products")).fetchall()` → `[]`

Untuk `test_loads_facts_basic()` :

```
mock_conn.execute.return_value.fetchall.side_effect = [[]]
# Hanya satu pemanggilan fetchall() karena load_fact() hanya query fact_sales satu kali
```

11.6 `assert` — Verifikasi Perilaku

```
assert mock_conn.execute.call_count == 2
```

`assert` adalah pernyataan yang **memverifikasi** bahwa suatu kondisi bernilai `True`. Jika kondisi `False`, test akan **gagal** dengan `AssertionError`.

`mock_conn.execute.call_count` — Properti MagicMock yang menghitung berapa kali method `.execute()` dipanggil.

```
# Dalam test_load_dimensions_basic():
# mock_conn.execute dipanggil 2 kali:
# 1. conn.execute(text("SELECT CustomerID FROM dim_customers"))
# 2. conn.execute(text("SELECT StockCode FROM dim_products"))
# Jadi call_count == 2 → assert LULUS ✓

# Dalam test_loads_facts_basic():
# mock_conn.execute dipanggil 1 kali:
# 1. conn.execute(text("SELECT InvoiceNo, StockCode FROM fact_sales"))
# Jadi call_count == 1 → assert LULUS ✓
```

11.7 `pd.option_context("mode.chained_assignment", None)` — Mengapa Diperlukan?

```
with pd.option_context("mode.chained_assignment", None):
    load_dimensions(df, engine)
```

Masalah: Di dalam `load_dimensions()`, ada baris:

```
dim_customers["CustomerID"] = dim_customers["CustomerID"].astype(int)
```

Baris ini melakukan **chained assignment** — mengubah kolom dari DataFrame yang merupakan **slice** (potongan) dari DataFrame asli. Pandas mengeluarkan **warning** (`SettingWithCopyWarning`) karena operasi ini ambigu — apakah kita mengubah slice atau DataFrame asli?

`pd.option_context("mode.chained_assignment", None)` mematikan warning ini di dalam blok `with`. Ini dilakukan di test karena:

1. Kita **tahu** bahwa kode ini aman (tidak menyebabkan bug)
2. Warning ini mengganggu output test
3. Ini hanya mematikan warning **sementara** (hanya di dalam blok `with`)

12. 🗄 Konsep Database Yang Digunakan

12.1 DDL (Data Definition Language)

DDL adalah perintah SQL yang digunakan untuk **mendefinisikan struktur** database — membuat, mengubah, atau menghapus tabel.

Perintah	Fungsi	Contoh
<code>CREATE TABLE</code>	Membuat tabel baru	<code>CREATE TABLE dim_customers (...)</code>
<code>ALTER TABLE</code>	Mengubah struktur tabel	<code>ALTER TABLE dim_customers ADD COLUMN email VARCHAR(100)</code>
<code>DROP TABLE</code>	Menghapus tabel	<code>DROP TABLE dim_customers</code>
<code>CREATE TABLE IF NOT EXISTS</code>	Membuat tabel hanya jika belum ada	<code>CREATE TABLE IF NOT EXISTS dim_customers (...)</code>

Digunakan di `init_db()`

Di kode kita: DDL digunakan di fungsi `init_db()` untuk membuat 3 tabel (`dim_customers`, `dim_products`, `fact_sales`).

12.2 DML (Data Manipulation Language)

DML adalah perintah SQL yang digunakan untuk **memanipulasi data** di dalam tabel.

Perintah	Fungsi	Contoh
<code>INSERT</code>	Menambah data baru	<code>INSERT INTO dim_customers VALUES (12345, 'UK')</code>
<code>SELECT</code>	Membaca/mengambil data	<code>SELECT CustomerID FROM dim_customers</code>
<code>UPDATE</code>	Mengubah data existing	<code>UPDATE dim_customers SET Country='US' WHERE CustomerID=12345</code>
<code>DELETE</code>	Menghapus data	<code>DELETE FROM dim_customers WHERE CustomerID=12345</code>

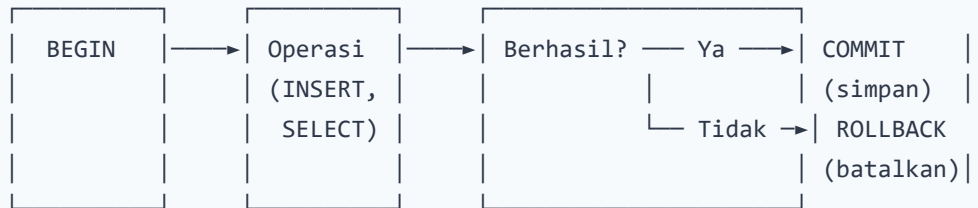
Di kode kita:

- `SELECT` digunakan di `load_dimensions()` dan `load_fact()` untuk mengecek data existing
- `INSERT` dilakukan secara implisit melalui `to_sql()` dari Pandas

12.3 Transactions — BEGIN, COMMIT, ROLLBACK

Perintah	Fungsi	Digunakan Di
<code>BEGIN</code>	Memulai transaksi baru	<code>conn.begin()</code> di <code>init_db()</code> , <code>engine.begin()</code> di <code>load_dimensions()</code> & <code>load_fact()</code>
<code>COMMIT</code>	Menyimpan perubahan secara permanen	<code>conn.commit()</code> di <code>init_db()</code> , otomatis di <code>engine.begin()</code>
<code>ROLLBACK</code>	Membatalkan semua perubahan	<code>conn.rollback()</code> di <code>init_db()</code> , otomatis di <code>engine.begin()</code>

Alur Transaksi:



12.4 Constraints — PRIMARY KEY, FOREIGN KEY

Constraint	Fungsi	Aturan
<code>PRIMARY KEY</code>	Mengidentifikasi baris secara unik	Harus UNIQUE dan NOT NULL
<code>FOREIGN KEY</code>	Menghubungkan tabel	Nilai harus ada di tabel yang dirujuk
<code>NOT NULL</code>	Kolom tidak boleh kosong	Implisit pada PRIMARY KEY
<code>UNIQUE</code>	Nilai harus unik di seluruh kolom	Implisit pada PRIMARY KEY

12.5 Data Types Mapping: Python → SQL

Python Type	Pandas dtype	SQL Type	Contoh
<code>int</code>	<code>int64</code>	<code>INT</code>	<code>12345</code>
<code>float</code>	<code>float64</code>	<code>REAL</code>	<code>2.55</code>
<code>str</code>	<code>object</code>	<code>VARCHAR(n)</code>	<code>"UK"</code>
<code>datetime</code>	<code>datetime64</code>	<code>DATETIME</code> / <code>TIMESTAMP</code>	<code>2010-12-01 08:26:00</code>
<code>bool</code>	<code>bool</code>	<code>BOOLEAN</code>	<code>True</code> / <code>False</code>

💡 Ketika menggunakan `to_sql()`, Pandas secara otomatis melakukan mapping tipe data ini. Tapi penting untuk memastikan tipe data sudah benar di DataFrame sebelum `to_sql()` dipanggil (itulah mengapa kita memanggil `.astype(int)`).

13. Konsep Python Yang Digunakan

13.1 Context Managers (`with` Statement)

Context manager memastikan bahwa **resource** (file, koneksi database, dll.) dibersihkan dengan benar setelah selesai digunakan, bahkan jika terjadi error.

```
# Di kode kita, digunakan di 3 tempat:

# 1. init_db() – koneksi database
with engine.connect() as conn:
    # conn otomatis ditutup setelah blok ini

# 2. load_dimensions() – transaksi database
with engine.begin() as conn:
    # conn otomatis commit/rollback setelah blok ini

# 3. Test file – pandas option
with pd.option_context("mode.chained_assignment", None):
    # Setting otomatis dikembalikan setelah blok ini
```

Cara kerja context manager:

```
# with X as Y:
#     ...

# Setara dengan:
Y = X.__enter__()      # Dipanggil saat masuk blok 'with'
try:
    ...                # Kode di dalam blok
finally:
    X.__exit__()       # Dipanggil saat keluar blok (selalu, bahkan jika error)
```

13.2 Set Operations dan Set Comprehension

Set adalah koleksi data yang **tidak terurut** dan **tidak ada duplikat**.

```
# Set comprehension (membuat set dengan syntax ringkas):
exists_customers = {row[0] for row in exists_customers_raw}

# Setara dengan:
exists_customers = set()
for row in exists_customers_raw:
    exists_customers.add(row[0])
```

Operasi set yang digunakan:

```
# 'in' – cek keanggotaan (0(1) untuk set!)
12345 in {12345, 67890} # True
99999 in {12345, 67890} # False

# 'not in' – negasi keanggotaan
12345 not in {12345, 67890} # False
99999 not in {12345, 67890} # True
```

13.3 Tuple Unpacking

Tuple adalah koleksi data yang **immutable** (tidak bisa diubah) dan **terurut**.

```
# Di kode kita, tuple digunakan untuk composite key:
exists_sales = {(str(row[0]), str(row[1])) for row in exists_sales_raw}

# Setiap elemen: (InvoiceNo, StockCode)
# Contoh: ("INV001", "85123A")

# Tuple bisa digunakan sebagai elemen set (karena immutable)
# List TIDAK bisa digunakan sebagai elemen set (karena mutable)
```

13.4 Boolean Indexing/Masking

Boolean indexing adalah teknik Pandas untuk memfilter DataFrame menggunakan array/Series of boolean values.

```
# Cara 1: Menggunakan Pandas boolean Series (di load_dimensions):
mask = ~dim_customers["CustomerID"].isin(exists_customers)
new_customers = dim_customers[mask]

# Cara 2: Menggunakan Python list of booleans (di load_fact):
mask = [t not in exists_sales for t in sales_tuples]
new_sales = fact_sales[mask]

# Kedua cara menghasilkan output yang sama:
# Baris dengan mask True → dipilih
# Baris dengan mask False → dibuang
```

13.5 List Comprehension

List comprehension adalah cara ringkas untuk membuat list baru dari iterable yang sudah ada.

```
# Syntax: [expression for item in iterable if condition]

# Di kode kita:
mask = [t not in exists_sales for t in sales_tuples]

# Setara dengan:
mask = []
for t in sales_tuples:
    mask.append(t not in exists_sales)
```

13.6 `zip()` Function

`zip()` menggabungkan elemen-elemen dari beberapa iterable berdasarkan posisi/indeks.

```
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]

# zip(names, ages) menghasilkan:
# ("Alice", 25), ("Bob", 30), ("Charlie", 35)

# Di kode kita:
sales_tuples = list(zip(
    fact_sales["InvoiceNo"].astype(str), # Kolom 1
    fact_sales["StockCode"].astype(str)  # Kolom 2
))
# Menghasilkan: [("INV001", "85123A"), ("INV001", "71053"), ...]
```

13.7 Operator (Tilde/Negasi)

Dalam konteks Pandas,  adalah operator **negasi boolean** yang membalik setiap elemen dalam boolean Series.

```
import pandas as pd

s = pd.Series([True, False, True, False])
print(~s)
# Output:
# 0    False
# 1     True
# 2    False
# 3     True

# Di kode kita:
# dim_customers["CustomerID"].isin(exists_customers)
# → [True, True, False] (True = sudah ada di DB)

# ~dim_customers["CustomerID"].isin(exists_customers)
# → [False, False, True] (True = BELUM ada di DB = data baru)
```

⚠ **Penting:** Jangan bingung  (bitwise NOT) dengan  (logical NOT). Dalam Pandas:

-  → bekerja elemen per elemen ✓
-  → error! (`ValueError: The truth value of a Series is ambiguous`) ✗

14. Idempotency — Konsep Penting dalam ETL

14.1 Apa Itu Idempotency?

Idempotency (idempoten) adalah properti di mana suatu operasi bisa **dijalankan berkali-kali** dengan **hasil yang sama** seperti dijalankan sekali. Dalam konteks ETL, ini berarti: menjalankan pipeline 1 kali atau 10 kali menghasilkan **database yang sama** — tidak ada duplikasi data.

Tanpa Idempotency:	Dengan Idempotency:
Jalankan 1x → 1000 baris	Jalankan 1x → 1000 baris
Jalankan 2x → 2000 baris ✗ DUPLIKAT!	Jalankan 2x → 1000 baris ✓ SAMA!
Jalankan 3x → 3000 baris ✗ DUPLIKAT!	Jalankan 3x → 1000 baris ✓ SAMA!

14.2 Bagaimana Kode Ini Mencapai Idempotency?

Kode kita mencapai idempotency melalui **dua mekanisme**:

Mekanisme 1: DDL Idempotent — `CREATE TABLE IF NOT EXISTS`

```
CREATE TABLE IF NOT EXISTS dim_customers (...)
```

- Jika tabel **belum ada** → buat tabel baru ✓
- Jika tabel **sudah ada** → tidak melakukan apa-apa (tidak error, tidak mengubah tabel) ✓

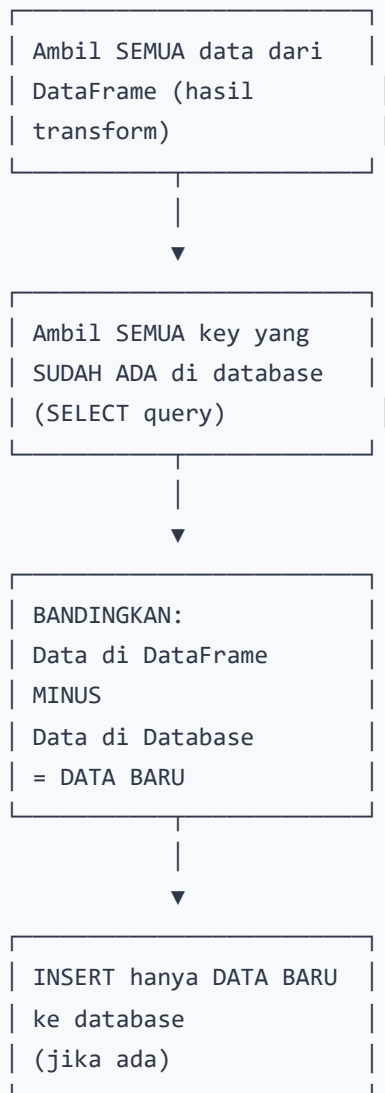
Ini membuat `init_db()` aman dijalankan berkali-kali tanpa efek samping.

Mekanisme 2: DML Idempotent — Pengecekan Data Existing Sebelum Insert

```
# Di load_dimensions():
exists_customers = {row[0] for row in exists_customers_raw} # Ambil data existing
new_customers = dim_customers[~dim_customers["CustomerID"].isin(exists_customers)] # Filter hanya baru

# Di load_fact():
exists_sales = {(str(row[0]), str(row[1])) for row in exists_sales_raw} # Ambil data existing
new_sales = fact_sales[[t not in exists_sales for t in sales_tuples]] # Filter hanya baru
```

Alur logikanya:

**Contoh skenario:**

```

=== Jalankan Pipeline ke-1 ===
DataFrame: CustomerID [12345, 67890, 11111]
Database: CustomerID [] ← kosong
Baru: CustomerID [12345, 67890, 11111] ← semua baru
→ INSERT 3 customer

=== Jalankan Pipeline ke-2 (data yang SAMA) ===
DataFrame: CustomerID [12345, 67890, 11111]
Database: CustomerID [12345, 67890, 11111] ← sudah terisi dari pipeline ke-1
Baru: CustomerID [] ← tidak ada yang baru
→ INSERT 0 customer (log: "No new customers to load")

=== Jalankan Pipeline ke-3 (ada data baru) ===
DataFrame: CustomerID [12345, 67890, 11111, 99999]
Database: CustomerID [12345, 67890, 11111]
Baru: CustomerID [99999] ← hanya 1 baru
→ INSERT 1 customer

```

14.3 Ringkasan Mekanisme Idempotency

Bagian Kode	Jenis	Mekanisme	Idempotent?	
<code>init_db()</code>	DDL	<code>CREATE TABLE</code>	Ya	— CREATE
<code>load_dimensions()</code>	DML	INSERT	Ya	— Cek existing keys, filter baru, insert baru
<code>load_fact()</code>	DML	INSERT	Ya	— Cek existing composite keys, filter baru, insert baru

15. Ringkasan & Poin-Poin Penting

15.1 Checklist Apa yang Sudah Dipelajari

Selamat! 🎉 Jika kamu sudah membaca seluruh dokumen ini, kamu sudah mempelajari:

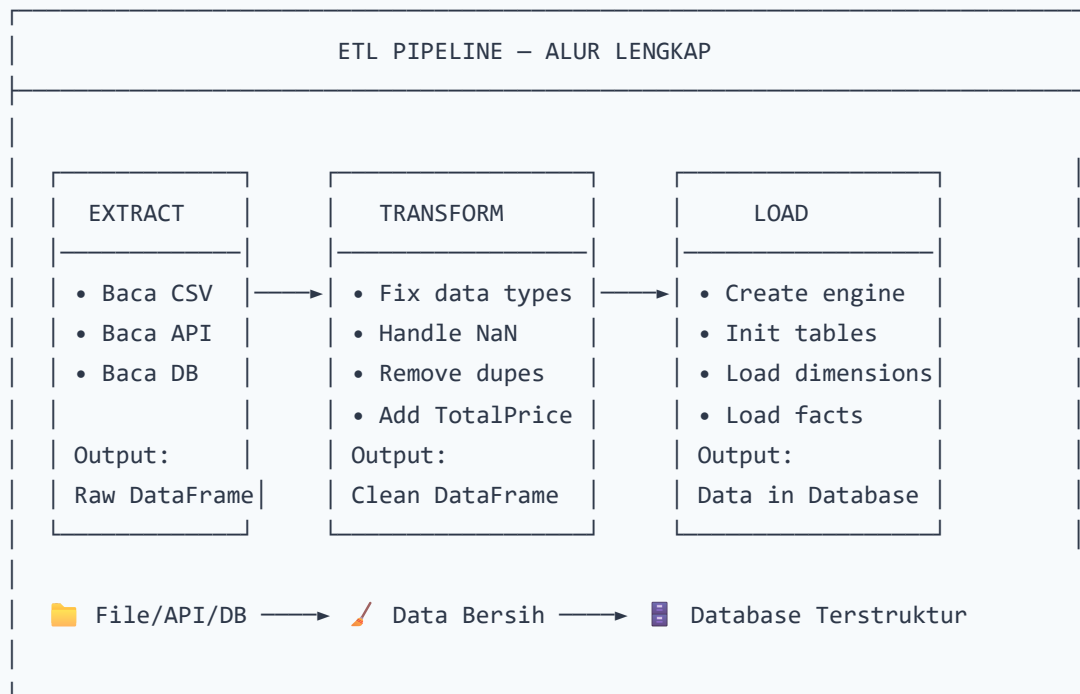
- ☒ **Apa itu Load** dalam konteks ETL dan mengapa ini fase paling kritikal
- ☒ **Star Schema** — Dimension Tables vs Fact Tables dan cara kerjanya
- ☒ **PRIMARY KEY** — constraint unik untuk identifikasi baris
- ☒ **FOREIGN KEY** — constraint relasional antar tabel
- ☒ **COMPOSITE PRIMARY KEY** — primary key dari dua kolom
- ☒ **SQLAlchemy** — library Python untuk interaksi database (Core vs ORM)
- ☒ **Connection String** — format URI untuk berbagai jenis database
- ☒ **Connection Pool** — mekanisme efisiensi koneksi database
- ☒ `engine.connect()` — koneksi tanpa transaksi otomatis

- [] ✓ `engine.begin()` — koneksi dengan transaksi otomatis
- [] ✓ **ACID Properties** — Atomicity, Consistency, Isolation, Durability
- [] ✓ **commit() vs rollback()** — menyimpan vs membatalkan perubahan
- [] ✓ **Context Manager** (`with` statement) — auto-close resource
- [] ✓ **DDL** (CREATE TABLE) dan **DML** (SELECT, INSERT)
- [] ✓ `to_sql()` — method Pandas untuk menulis DataFrame ke database
- [] ✓ **Deduplikasi** — single key vs composite key
- [] ✓ **Set operations** — mengapa set lebih cepat dari list untuk lookup
- [] ✓ **Boolean indexing** — memfilter DataFrame dengan mask boolean
- [] ✓ `zip()` — menggabungkan kolom menjadi tuple
- [] ✓ **Operator** `~` — negasi boolean di Pandas
- [] ✓ **Idempotency** — menjalankan ulang tanpa duplikasi
- [] ✓ **Unit Testing** dengan MagicMock — testing tanpa database asli

15.2 Tips dan Best Practices untuk Load Phase

| No | Tips | Penjelasan | |----|-----|-----| | 1 | 🛡️ **Jangan hardcode credentials** | Gunakan environment variables atau file `.env` | | 2 | 📦 **Selalu gunakan transaksi** | Wrap operasi DML dalam `engine.begin()` untuk keamanan data | | 3 | 🔄 **Buat kode idempotent** | Gunakan `IF NOT EXISTS` dan cek data existing sebelum insert | | 4 | 📄 **Gunakan logging** | Log setiap langkah penting untuk monitoring dan debugging | | 5 | 🧪 **Tulis unit test** | Mock database agar test cepat dan terisolasi | | 6 | ⌚ **Load dimensions sebelum facts** | Foreign Key constraint mengharuskan urutan ini | | 7 | 🚫 **Jangan gunakan `if_exists="replace"`** | Ini menghapus seluruh data yang sudah ada! Gunakan `"append"` | | 8 | 🗺️ **Gunakan `index=False` di `to_sql()`** | Hindari index DataFrame menjadi kolom tambahan di database | | 9 | 🔍 **Validasi tipe data sebelum insert** | Pastikan `.astype()` sudah dipanggil sebelum `to_sql()` | | 10 | 🏗️ **Gunakan Star Schema** | Pisahkan data master (dimensi) dari data transaksional (fakta) |

15.3 Alur Lengkap ETL Pipeline — Dari Awal Sampai Akhir



 **Catatan Akhir:** Dokumen ini adalah bagian dari seri pembelajaran ETL Pipeline. Pastikan kamu juga mempelajari fase Extract dan Transform untuk pemahaman yang lengkap. Selamat belajar! 